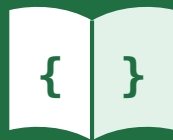

Module 19 — Native Image and Performance

Exercises

Spring Boot Edu



2026-09-25

Contents

How to Work	2
Exercise 1 — Make Reflective Code Native-Ready (Medium)	2
Exercise 2 — Which Beans Make the Startup Slow? (Medium)	3
Exercise 3 — Measure and Fill in the Table (Easy)	3
Extra Challenge (Optional)	4

How to Work

1. The starter code is in `modules/19-native-performance/exercise/`. Find the TODO comments.
2. Exercises 1 and 2 need no Docker and no GraalVM: the tests check the hints and the startup steps on the JVM. Exercise 3 is a measurement with the lesson application.
3. Run the tests:

```
./mvnw -Pexercises -pl modules/19-native-performance/exercise -am test
```

4. The exercise is done when all tests are green (and, for exercise 3, the table is filled in).
5. If you get stuck, read the hints first, then the solution in `modules/19-native-performance/solution/`.

Exercise 1 — Make Reflective Code Native-Ready (Medium)

Goal: `ExportService` works on the JVM. In a native image, three things would be missing: the export format that is created by class name, the file `export/header.txt`, and the JSON binding of `ImportedBook`. Add the hints.

Tasks (package `exercisel`):

- TODO 1a (`ExportHints`) — allow creating `CsvExport` and `MarkdownExport` by reflection, and include `export/header.txt`.
- TODO 1b (`ExportService`) — register `ExportHints` for the service.
- TODO 1c (`ExportService`) — tell Spring that `ImportedBook` is bound from JSON.

Hints:

- Lesson section 3.4: `hints.reflection().registerType(..., MemberCategory.INVOKE_DECLARED_CONSTRUCTORS)` and `hints.resources().registerPattern(...)`.
- `@ImportRuntimeHints(ExportHints.class)` works on any bean class.
- `@RegisterReflectionForBinding(ImportedBook.class)` registers what Jackson needs (constructor, accessors).
- `ExerciselTest` runs Spring's real AOT processing (`ApplicationContextAotGenerator`) for the service, as `process-aot` does at build time.

Acceptance criteria: all 3 tests in `ExerciselTest` pass.

Estimated time: 25 minutes

Exercise 2 — Which Beans Make the Startup Slow? (Medium)

Goal: the application records its startup steps with `BufferingApplicationStartup` (see `NativePerformanceApplication`). Write a report of the slowest bean creations. `SlowCatalogWarmup` needs 300 ms on purpose.

Tasks (`exercise2.StartupReport.slowestBeans`):

- TODO 2a — take the recorded steps named `spring.beans.instantiate`.
- TODO 2b — turn each into a `BeanTiming`. The bean name is the step's tag `beanName`.
- TODO 2c — sort the slowest first and return at most limit.

Hints:

- `startup.getBufferedTimeline().getEvents()` returns `TimelineEvents` with `getStartupStep()` and `getDuration()`.
- `event.getStartupStep().getTags()` is an `Iterable<StartupStep.Tag>` with `getKey()` and `getValue()`.
- `Comparator.comparing(BeanTiming::duration).reversed()`.
- With Actuator, the same data is available at `/actuator/startup`.

Acceptance criteria: `Exercise2Test` passes.

Estimated time: 25 minutes

Exercise 3 — Measure and Fill in the Table (Easy)

Goal: see with your own numbers what AOT processing, the AOT cache and a native image change.

Tasks:

1. Start PostgreSQL (`docker compose --profile postgres up -d`) and build the native image once (lesson section 3.5).
2. Run `modules/19-native-performance/compare-startup.sh` 5.
3. Copy your numbers into the table and answer the questions.

Variant	Start ms	RSS MB
JVM		
JVM + Spring AOT		
JVM + Spring AOT + AOT cache		
Native image		

- Which step gives the largest gain in startup time? Which one in memory?
- Run the AOT cache variant without the training run (delete `target/startup/app.aot`). What happens?
- What do you give up with the native image (build time, peak performance, reflection)? When is it worth it?

Acceptance criteria: the table is filled in, and each question has a two-sentence answer.

Estimated time: 30 minutes (plus the native build)

Extra Challenge (Optional)

Add `spring-boot-starter-actuator` to the lesson, expose `/actuator/startup` and compare its output with your `StartupReport`. Then build the native image with the actuator and check whether the endpoint still works.